



香港都會大學
HONG KONG
METROPOLITAN
UNIVERSITY

School of Science and Technology
MCOMPF - Master of Computing
COMP 8960SEF Capstone Project

An Intelligent Weather Forecast Notification System: WeatherWiseBot

Student Name: HE Xue
SID: 13927408
Supervisor: Dr. Jeff Au Yeung

May. 2026

Abstract

Climate change has increased the frequency of extreme weathers, making timely and personalised weather information crucial for daily life. This project initiated the development of **WeatherWiseBot**, an intelligent weather forecast notification system that delivers weather updates, smart clothing recommendations, and severe weather warnings directly to users through Telegram.

The system integrates the OpenWeather API for real-time meteorological data, a rule-based recommendation engine for outfit suggestions, and a Telegram Bot for instant message delivery. A trip planning module further enhances the system by comparing weather conditions between departure and arrival cities, providing travellers with targeted advice.

The prototype has been deployed as a web application with four functional pages: Home, Weather Query, Telegram Push, and Trip Weather Push. Two additional features: scheduled daily weather push and a user login system are planned for the next development phase.

Keywords: Weather forecast, Telegram Bot, Streamlit, Open Weather, Python

Table of Contents

Chapter 1. Introduction

- 1.1 Overview
- 1.2 Project Aim
- 1.3 Project Objectives
- 1.4 Value Propositions of this Projects

Chapter 2. Literature Review

- 2.1 Review of Related Solutions for the Problem
- 2.2 Highlight of the Proposed Solution

Chapter 3. Methodology

- 3.1 Requirements, Supporting Technologies, and Technical Gap
- 3.2 System Design
- 3.3 Implementation Issue
- 3.4 Evaluation Methods and Design

Appendix

- A. Overview of Project Progress
- B. Revised Project Plan

An Intelligent Weather Forecast Notification System: WeatherWiseBot

Chapter 1. Introduction

1.1 Overview

(The **Overview** of this project has remained unchanged.) With the increasing frequency of extreme weather events driven by climate change, personalised and timely weather information has become essential for daily life. Storms, heavy snow, and sudden rains happen more often, making weather more unpredictable.

Traditional weather apps are basic — users open them, check the forecast, and that's it. They do not send smart alerts based on user location. This creates a significant gap: users need a system that actively pushes relevant information without requiring constant interaction. This project presents WeatherWiseBot, an intelligent weather forecast notification system that delivers proactive, personalized weather updates to users.

1.2 Project Aim

(The **Project Aim** of this part remained unchanged.) The aim of this project is to develop WeatherWiseBot, an intelligent weather forecast notification system that makes weather information more personal and proactive, helping individuals make better daily plans while contributing to societal benefits such as fewer accidents and reduced costs.

(The **Project Aim** of this project is changed to the followings:) During development, a design change was made to the notification delivery channel. The SMS notification module has been removed and replaced with Telegram Bot for weather message delivery. The reason behind the change is due to that the SMS service providers (Twilio or Submail) require paid subscriptions, mandatory enterprise verification, which add complexity and inconvenient for my capstone project. Telegram Bot, in contrast, is free, globally accessible, and easier to integrate. A discussion was held between me and the supervisor on 22 April 2026 and it was agreed that Telegram Bot would replace SMS as the notification channel.

1.3 Project Objectives

(The **Project Objectives** of this project are updated to the followings:)

The objectives of this project are to:

1) Design a user-friendly web interface using Streamlit

This objective has been 70% completed. The interface now consists of four pages: Home, Weather Query, Telegram Push and Trip Weather push.

2) Integrate a weather API

Integrate a weather API for real-time, location-specific forecasts, warnings, and indices across regions. This objective has been completed through the integration of the OpenWeather API.

3) Implement a Telegram-based notification system

This objective is revised from the SMS notifications and has been completed. The system delivers weather forecasts directly to users' Telegram accounts via the WeatherWiseBot.

4) Bad Weather Checking

Manual severe weather checking has been implemented and integrated into the Weather Query and Telegram Push pages. The system detects extreme heat, extreme cold, rain, and strong wind conditions.

5) Rule-based clothing recommendations

This objective has been completed. The engine groups temperatures into six ranges and adds contextual adjustments for rain, snow, thunderstorms, and strong wind.

6) Scheduled Weather Push System (New Objectives, planned for next stage)

Implement a weather push system that allows users to configure a specific daily push time. Then the system will automatically deliver weather reports to Telegram Bot.

7) Implement a user login and profile management system (New Objectives, planned for next stage)

Implement a user login and profile management system that stores personal information and frequently used friend Telegram IDs, eliminating the need to repeatedly enter contact details for every notification.

1.4 Value Propositions of this Project

(The **Value Propositions** of this part remained unchanged.) For individuals, WeatherWiseBot saves time and improves safety. Commuters, travelers, and outdoor enthusiasts receive practical, schedule-aware alerts (e.g., outfit tips before a hike or flight delay warnings).

On a societal level, fewer weather-related accidents mean lower costs and better public preparedness. Stakeholders include end-users, meteorological data providers, and potentially healthcare systems (reduced cold-related illnesses through better clothing choices). Overall, WeatherWiseBot promotes a proactive approach to weather awareness in an era of climate uncertainty.

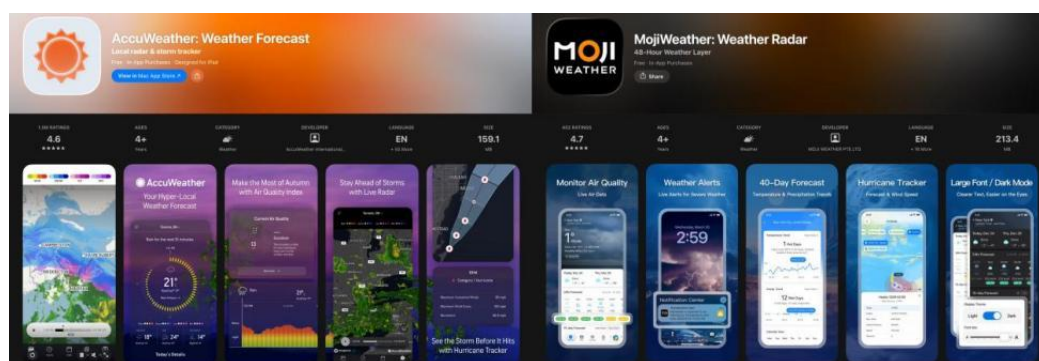
Chapter 2. Literature Review

2.1 Review of Related Solutions for the Problem

(The **Review of Related Solutions for the Problem** of this part remained unchanged.)

Numerous weather applications and notification systems exist, providing valuable lessons for this project. Popular mobile apps like AccuWeather, MojiWeather, The Weather Channel, and WeatherBug dominate the market with features such as local forecasts, radar visualization, and push notifications for severe weather.

AccuWeather offers "MinuteCast" for minute-by-minute precipitation predictions, while MojiWeather provides accurate real-time weather updates, air quality indexes pushes and severe weather warnings for millions of locations. These systems have high user adoption due to intuitive interfaces than traditional weather forecast.



Specialized apps focus on niche personalization. Weather Fit provides weather-based clothing recommendations, suggesting layers, accessories, or rain gear based on temperature and precipitation. Apps like Carrot Weather add humorous notifications and basic personalization, while wardrobe tools integrate weather for outfit planning from user's digital closets.



(The **Value Propositions** of this project are updated to the followings:)

The literature review has been updated to reflect that I changed the SMS delivery mechanism to Telegram Bot notifications. Most existing apps still rely only on in-app push notifications, which users can easily disable or miss when their phone is on silent. In addition, recommendations in those apps are usually too generic. A discussion was held between me and the supervisor on 22 April 2026 and we agreed to revise the literature review to match the new project scope: focus on Telegram Bot as the notification channel.

2.2 Highlight of the Proposed Solution

(The **Highlight of the Proposed Solution** of this project are updated to the followings:)

WeatherWiseBot differentiates itself by focusing on personalization and proactive delivery. Key highlights include:

- (1) Region-Specific forecasts with outfit recommendations.
- (2) Severe Weather Alert.
- (3) Rule-based 'if-condition, then-result' outfit suggestions.
- (4) Telegram Bot push notification.
- (5) Trip Weather Push

The notification delivery is now handled via Telegram Bot instead of SMS. The system now offers four core functions: weather query, outfit recommendation, telegram push notification, and trip weather push.

The reason behind the change is due to that Telegram Bot API provides free and global message delivery without the barriers of commercial SMS services. A discussion was held between me and the supervisor on 22 April 2026 and it was concluded that the current four-function system would be more deliverable and testable.



Telegram Bot (WeatherWiseBot)

Chapter 3. Methodology

3.1 Requirements, Supporting Technologies, and Technical Gap

(The Requirements, Supporting Technologies, and Technical Gap of this project are updated to the followings:) The project adopts an agile development methodology, combined with waterfall model for clear requirement definition. A top-down design approach is used, starting from high-level features and refining stepwise into individual modules.

3.1.1 Functional Requirements

- 1) Support multi-city weather query via Streamlit web interface.
- 2) Retrieve real-time forecasts and warnings from OpenWeather API.
- 3) Provide manual weather push via Telegram Bot.
- 4) Detect severe conditions on demand and display alerts.
- 5) Provide rule-based clothing recommendations.
- 6) **(Further improvement)** Implement a weather push system that allows users to configure a specific daily push time.
- 7) **(Further improvement)** Implement a user login and profile management system.

3.1.2 Non-functional Requirements

- 1) Reliability: API failure handling with user-friendly error messages (e.g., "City not found" instead of a crash).
- 2) Performance: API calls < 5 seconds, Telegram delivery < 10 seconds.
- 3) Usability: Intuitive Streamlit interface with clear navigation.
- 4) Scalability: Start with single-user, extensible to multiple users.

3.1.3 Supporting Technologies

- 1) Python as the core language for development.
- 2) Streamlit as the web interface framework.
- 3) OpenWeather API for weather data.
- 4) Telegram Bot replaces Twilio/Submail SMS service as the notification delivery mechanism.
- 5) Pandas for data processing.
- 6) SQLite database and automated scheduler are planned for the next stage.

3.1.4 Technical gaps

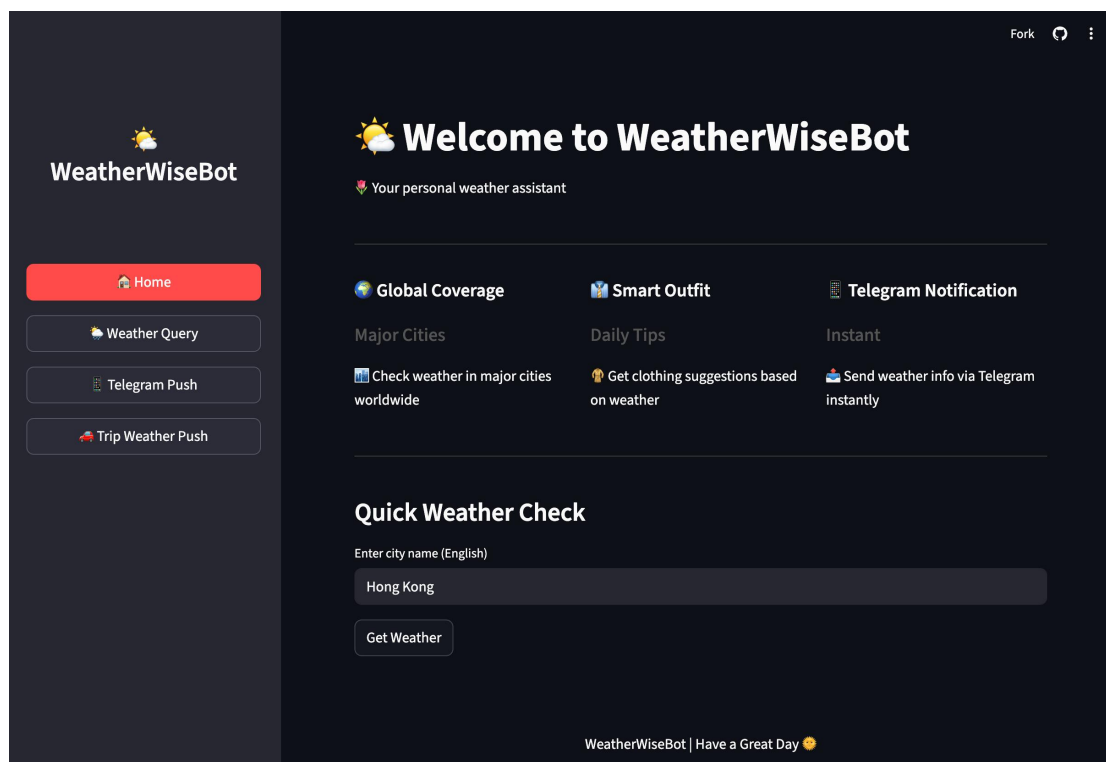
- 1) Clothing recommendations are currently rule-based rather than AI-driven.
- 2) Dependency on external API availability (Openweather API, Telegram Bot API).
- 3) Telegram Bot requires users to initiate contact before messages can be delivered.
- 4) There is no persistent storage in current system.
- 5) Scheduled push and user login features are not yet implemented.

3.2 System Design

(The **System Design** of this project are updated to the followings:)

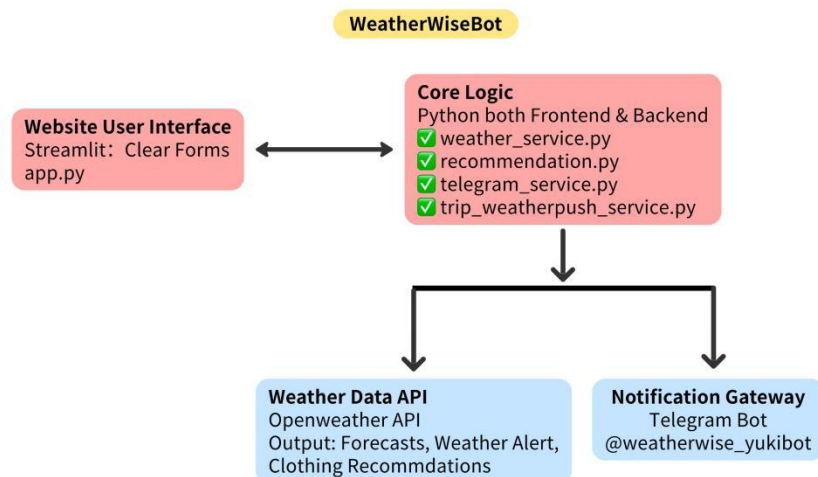
The system follows a layered architecture design:

- 1) **Presentation Layer:** Using streamlit web interface as a user-friendly frontend. Four pages (Home, Weather Query, Telegram Push, and Trip Weather Push) are connected via a sidebar navigation menu.



Streamlit Interface

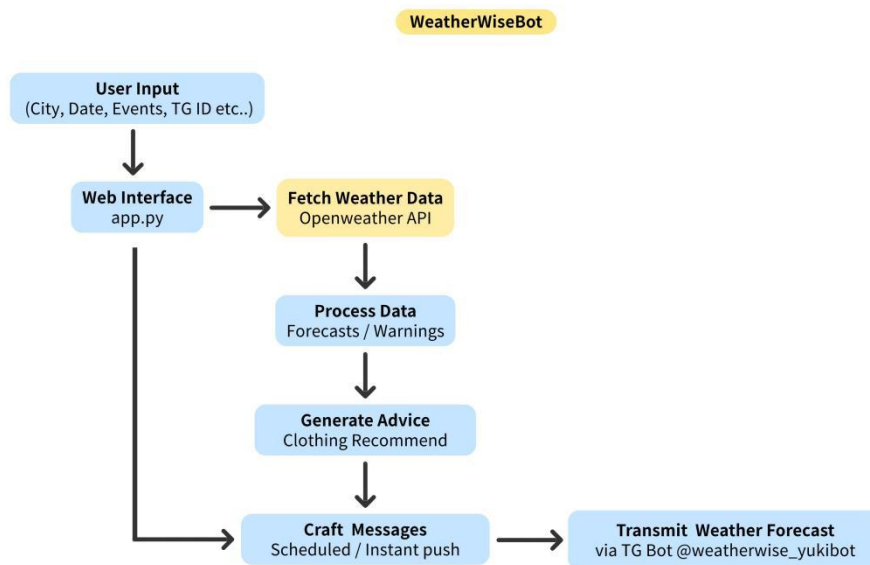
- 2) **Application Layer:** Python modules handling weather data fetching (weather_service.py), clothing recommendations (recommendation.py), and Telegram message building and dispatch (telegram_service.py) and trip weather reporting (trip_weatherpush_service.py).
- 3) **Data Layer:** Streamlit session state for temporary data persistence during a single browser session. SQLite is planned for the next phase to support user login and scheduled push.
- 4) **Integration Layer:** Requests for OpenWeather API; Requests for Telegram Bot API message sending. Weather data fetching is isolated from Telegram message sending, allowing independent testing of each component.



Component Diagram

Refers to the component diagram above, the WeatherWiseBot system comprises four interconnected components: Website User Interface (Streamlit), Core Logic Layer (four Python modules), Weather Data API (OpenWeather), and Notification Gateway (Telegram Bot).

The overall structure is almost the same as the initial report, with the Notification Gateway changed from Twilio/Submail SMS to Telegram Bot, and the trip_weatherpush_service.py added as a new module.



Data-Flow Diagram

Refers to the Data-Flow Diagram above, the data flow shows: User input → OpenWeather API call → Data processing → Display on Streamlit website or packaging into Telegram message → send to user-specified Telegram Chat ID via

WeatherWiseBot.

The system currently operates without persistent storage. This will change when the user login and scheduled push features are implemented, which will require SQLite database

3.3 Implementation Issue (New additions)

During the implementation phase, several practical issues were encountered and resolved:

3.3.1 SMS Service Remove to Telegram Bot

The original plan to use Submail or Twilio to send SMS messages. Submail requires a registered business entity. Twilio requires a verified paid account for reliable international messaging. Both services are impractical for my project. Telegram Bot API was selected as the replacement because it is free, requires only a simple bot creation via BotFather, supports global delivery.

3.3.2 OpenWeather API Handling

The OpenWeather API has some limits on how many requests can be made per minute. The weather data comes back in a nested JSON format, and the structure is different for current weather and forecast data. I had to handle various problems: some cities return unusual data, some fields can be missing, and sometimes the API simply does not respond in time.

3.3.3 Clothing Recommendation

The engine uses a rule-based approach: it checks conditions and triggers the corresponding result. The rules group temperatures into six ranges (below 0°C, 0–10, 10–18, 18–25, 25–32, above 32) and add extra suggestions based on weather conditions like rain, snow, or strong wind.

3.3.4 Scheduled Push (To be implemented)

Currently the system only supports instant push: users must manually click "Send" each time they want a weather report. The next stage I plan to introduce a scheduled push feature. At the configured time each day, the system will automatically fetch weather data for specific cities, and push the weather report to user's Telegram Bot.

3.3.5 User Login (To be implemented)

During meeting with tutor, a significant problem was identified: users must type their telegram ID for every push operation. For users who send weather reports frequently, this repetitive input is inconvenient. A login system will allow users to register an account, store their personal Telegram ID, and maintain a contact list of frequently used IDs (e.g., family, friends, colleagues). When sending notifications, users can simply select a recipient from the saved list instead of typing the ID manually.

3.4 Evaluation Methods and Design (New additions)

The evaluation of WeatherWiseBot will be conducted through the following methods:

3.4.1 Functional Testing

Each function is tested separately.

- 1) The weather queries return correct data for major cities.

- 2) The 5-day forecast shows the right daily high and low temperatures.
- 3) The clothing suggestions with different temperatures and weather conditions to make sure they make sense.
- 4) The telegram messages to real IDs to confirm delivery works.

3.4.2 Performance Testing

Measure OpenWeather API response times (target < 5 seconds) and Telegram Bot API message delivery latency (target < 10 seconds).

3.4.3 Usability Testing

A small group of users will be invited to use the website and give feedback. The evaluation will focus on:

- 1) Whether the navigation between pages is easy to follow
- 2) Whether the input fields and labels are clear enough for new users.
- 3) Whether the information layout (weather data, recommendations, alerts) is easy to read and understand.

The system has been deployed as a web application on Streamlit, accessible via a URL.

Appendix

A. Overview of Project Progress

The following table summarizes the current implementation status of each project component:

Component	Status	Notes
Streamlit Web Interface (Home, Weather Query, Telegram Push, Trip Weather Push)	70% Completed	Four pages implemented with sidebar navigation and session state.
OpenWeather API Integration	Completed	Current weather and 5-day forecast retrieval working.
Rule-based Clothing Recommendation	Completed	Temperature, condition, and wind-based suggestions implemented and tested.
Severe Weather Alert Checker	Completed	Available across all pages: Weather Query, Telegram Push, and Trip Weather Push.
Telegram Bot Notification Service	Completed	Message building and manual delivery via Telegram Bot API working.
SMS Notification Service (Submail/Twilio)	Removed	Removed due to paid subscription and verification requirements.

Scheduled Weather Push	To be implemented	A weather push function that timely delivers weather reports to Telegram Bot.
User Login & Profile System	To be implemented	A user login system that stores personal information and frequently used Telegram IDs, eliminating the need to repeatedly enter the ID for every notification.

B. Revised Project Plan

Based on the scope revision, the revised project plan is as follows:

Phase	Timeline	Activities
Core Implementation	March - June 2026	Streamlit UI development, OpenWeather API integration, clothing recommendation engine, Telegram Bot service, trip weather push module.
Testing	June - July 2026	Scheduled push feature, user login system, functional testing, performance testing, usability testing, bug fixing.
Finalization	July - August 2026	Final report writing, project presentation preparation, future extension.